Information Fusion 117 (2025) 102908



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus



Full length article



Obfuscation-resilient detection of Android third-party libraries using multi-scale code dependency fusion

Zhao Zhang, Senlin Luo, Yongxin Lu, Limin Pan *

Information System & Security and Countermeasures Experiments Center, Beijing Institute of Technology, Beijing, 100081, PR China

ARTICLE INFO

Keywords:
Third-party library
Code obfuscation
Identification of adjacent versions
Library dependency graph

ABSTRACT

Third-Party Library (TPL) detection is a crucial aspect of Android application security assessment, but it faces significant challenges due to code obfuscation. Existing methods often rely on single-scale features, such as class dependencies or instruction opcodes. This reliance can overlook critical dependencies, leading to incomplete library representation and reduced detection recall. Furthermore, the high similarity between a TPL and its adjacent versions causes overlaps in the feature space, reducing the accuracy of version identification. To address these limitations, we propose LibMD, a multi-scale code dependency fusion approach for TPL detection in Android apps. LibMD enhances library code representation by combining class reference syntax augmentation, cross-scale function mapping, and control flow reconstruction of basic blocks. It also extracts metadata dependencies and constructs a library dependency graph that integrates app-code similarity with multiple libraries. By applying Bayes' theorem to compute posterior probabilities, LibMD effectively evaluates the likelihood of TPL integration and improves the precision of library version identification. Experimental results demonstrate that LibMD outperforms state-of-the-art methods across diverse datasets, achieving robust TPL detection and accurate version identification, even under various obfuscation techniques.

1. Introduction

With the rapid expansion of mobile internet, the Android platform has become one of the most widely used operating systems globally [1,2]. Developers frequently rely on various third-party libraries (TPLs) to enhance development efficiency and accelerate time-to-market [3,4]. These libraries offer a range of functionalities such as network communication, image processing, and data storage, greatly enriching app capabilities. However, the reliance on TPLs also introduces potential security vulnerabilities and stability issues [5–8]. Recent reports [9] have highlighted problems like the embedding of malicious code and version inconsistencies, underscoring the critical need for robust detection methods for TPLs.

Android TPL detection refers to the identification and analysis of TPLs used within Android apps, ensuring the security, stability, and compatibility of these libraries. Initially, whitelist-based methods are employed to detect TPLs [10–13]. Since only commonly used libraries are included in the whitelist, some libraries are inevitably overlooked, and newly emerged libraries cannot be detected promptly. Furthermore, relying solely on package names for detection lacks resilience to code obfuscation techniques. Existing TPL detection methods utilize clustering [14,15] and similarity [16,17] to identify the TPLs invoked by apps. Clustering-based methods do not require prior collection of TPL files. The features of the library can be directly extracted through clustering algorithms to identify TPL in apps without prior knowledge. However, clustering algorithms generally require developers to specify the number of clusters or set appropriate parameters to achieve clustering results. In practice, determining suitable clustering parameters to perfectly distinguish between TPLs and their versions is highly challenging due to the diversity of TPL versions and code similarities. Developers also need to validate the clustering results to ensure their accuracy, which is labor-intensive and time-consuming. Additionally, such methods are often limited to identifying commonly used TPLs and may overlook niche or newly emerged libraries. The recall rate of the method depends on the number of input apps and the reuse rate of TPLs. In contrast, similarity-based methods require developers to collect TPL files to build a predefined feature database. The subtle differences and potential issues among different TPLs can be identified by comparing specific features (such as API interfaces, code structures, and functional implementations).

Apart from the aforementioned weaknesses, existing methods still have some limitations:

- Insufficient representation of library code under multi-type obfuscation. Existing methods [18–20] only extract features from class

* Corresponding author.
E-mail address: panlimin2016@gmail.com (L. Pan).

https://doi.org/10.1016/j.inffus.2024.102908
Received 17 September 2024; Received in revised form 21 December 2024; Accepted 24 December 2024
Available online 31 December 2024
1566-2535/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

code dependencies or instruction opcodes. The lack of potential critical dependencies at a single scale leads to weak integrity of library representation in code obfuscation scenarios, which reduces the recall rate of TPL detection [21]. For example, Zhang et al. [22] extract class dependencies to generate strict signatures and detect TPLs by computing the similarity between class signatures. However, functions within a class may call functions from other classes, creating "cross-scale dependencies" that involve both functions and classes, which cannot be effectively represented by a single scale. Furthermore, multi-type code obfuscation further reduces the distinguishability of library features, potentially leading to the omission of TPLs [23,24].

- Low discrimination between adjacent versions of TPLs. In recent works [25–27], the code similarity between a TPL and its adjacent versions results in a highly consistent distribution of library code in the feature space, which weakens the distinguishability between correct and incorrect versions. For example, Huang et al. [28] assess the probability of an app containing a given TPL using subset queries. However, adjacent library versions exhibit excessive code repetition, and the features of the correct version closely resemble those of adjacent versions and the app itself, leading to detection failure.

In this paper, a TPL detection method for Android apps using multi-scale code dependency fusion, LibMD, is proposed. Code dependency features of TPLs and apps at different scales are extracted, including class references, function invocations, and basic block control flows. Class reference syntax enhancement, function cross-scale mapping, and the instruction semantic fusion of basic block context are performed based on multiple code obfuscation types to improve the adequacy of library code representation; In addition, the library dependencies from TPL metadata are extracted, the code similarity of multiple libraries and apps are combined to construct a library dependency graph (LDG), the possibility of TPLs in apps can be calculated using Bayes' theorem [29] to achieve accurate identification of library versions. The effectiveness of LibMD in TPL detection and version identification is evaluated through a series of experiments conducted on three datasets. The main contributions of this paper are summarized as follows:

- A multi-scale dependency feature extraction method is proposed, which enhances feature coupling across different scales, reduces information loss caused by single-scale representation, and strengthens resilience against common code obfuscation techniques, including renaming and control flow flattening.
- A version identification method is developed that combines similarity and library dependency, effectively capturing subtle differences between library versions and enhancing the distinguishability of TPLs in the feature space.
- Extensive comparative and ablation experiments validate the superior performance of proposed method, achieving higher recall and precision in version identification while demonstrating robust resistance to various obfuscation techniques.

The remainder of this paper is organized as follows. In Section 2, we discuss related work on Android TPL detection. Section 3 presents the general framework and then introduces the methodology in detail. Section 4 evaluates the proposed approach through three sets of experiments. Section 5 discusses some potential limitations of the proposed approach and future work. Section 6 concludes the paper.

2. Related work

In this section, we review prominent research on Android TPL detection [30–34]. Based on their technical characteristics, existing methods can be categorized into three types: clustering-based detection, learning-based detection, and similarity-based detection.

(1) Clustering-based detections

LibD [35] extracts library candidates based on the reference and inheritance relations between Java classes and methods, classifies library candidates using feature hashing to handle code with obfuscated packages and names, and identifies TPL by clustering library candidates with equivalent features. LibRadar [36] extracts stable code features of all folders in the package structure and converts them into lightweight hashes. It identifies library candidates by multi-level clustering of a large number of apps and compares the hash features of a given app and library candidates to detect TPL. LibSift [37] uses package dependency graphs to decouple app code into primary modules and non-primary modules, where the primary modules define the core functions of the app and the non-primary module are provided by TPL. LibSift can detects TPL by identifying the primary modules. WuKong [38] filters out TPLs through automatic clustering, compares lightweight static semantic features to identify suspicious apps in the coarse-grained detection stage, and uses code block-level features to further judge the similarity of app pairs in the fine-grained detection stage.

(2) Learning-based detections

PEDAL [39] unpacks the app into Java bytecode, collects bytecode statistics to generate statistical vectors, and builds a classifier to divide the bytecode set into an ad library set and an app logic set. The rewriter performs binary rewriting on the bytecode set to achieve selective downgrade of ad library privileges. AdDetect [40] utilizes hierarchical clustering to identify and recover the primary and non-primary modules of an app. It extracts semantic features from each module and represents them as vectors, which are used to train an SVM classifier to detect ad libraries.

(3) Similarity-based detections

LibScout [25] extracts profiles from the original library based on class hierarchy information, uses profiles that are resilient to code obfuscation to detect libraries in apps, and performs similarity comparisons of TPLs through profile matching. CodeMatch [41] uses abstract code representations of several parts of the underlying bytecode to resist advanced obfuscation techniques. It uses fuzzy hash representations of app code to compare apps from different developers. PanGuard [42] uses the structure and content information of the package in the app to characterize the TPL, and determines the TPL and its version included in the app through the invariants in the mutation process, which can reduce the interference of code obfuscation, shrinkage and optimization. Orlis [18] can derive obfuscation-resilient code features in the inter-procedural structure and behavior (method level and class level) of an app, and utilize similarity digests to identify high-likelihood matches between a given obfuscated app and known libraries. ATVHunter [43] extracts the Control Flow Graph (CFG) of TPL as coarse-grained features to match potential TPLs in a predefined TPL database, and then extracts the opcodes in each basic block of the CFG as fine-grained features to identify the exact TPL version. LibID [26] generates a profile for each library and app binary and outputs the probability of a given library appearing in any given app by comparing the profiles for similarity. LIBPECKER [22] uses signature matching to calculate the similarity score between a given library and an app, and generates a strict signature for each class by extracting the internal class dependencies in the library. LIBPECKER introduces an adaptive class similarity threshold and a weighted class similarity score when calculating library similarity to improve tolerance to code customization and elimination.

In summary, the class dependency method only extracts code dependency features at a single class scale, which leads to the loss of cross-scale dependencies and false negatives in TPL detection under code obfuscation scenarios. For instance, functions located in different classes may have invocation relationships that span both class and function scales, and such dependencies cannot be represented solely by class-level features. In cases of multi-type code obfuscation, the lack of comprehensive dependency features reduces the similarity between the obfuscated app and the TPL, resulting in some TPLs being undetectable. Program execution instructions can be used to extract opcode features for identifying TPL versions. However, when code obfuscation removes

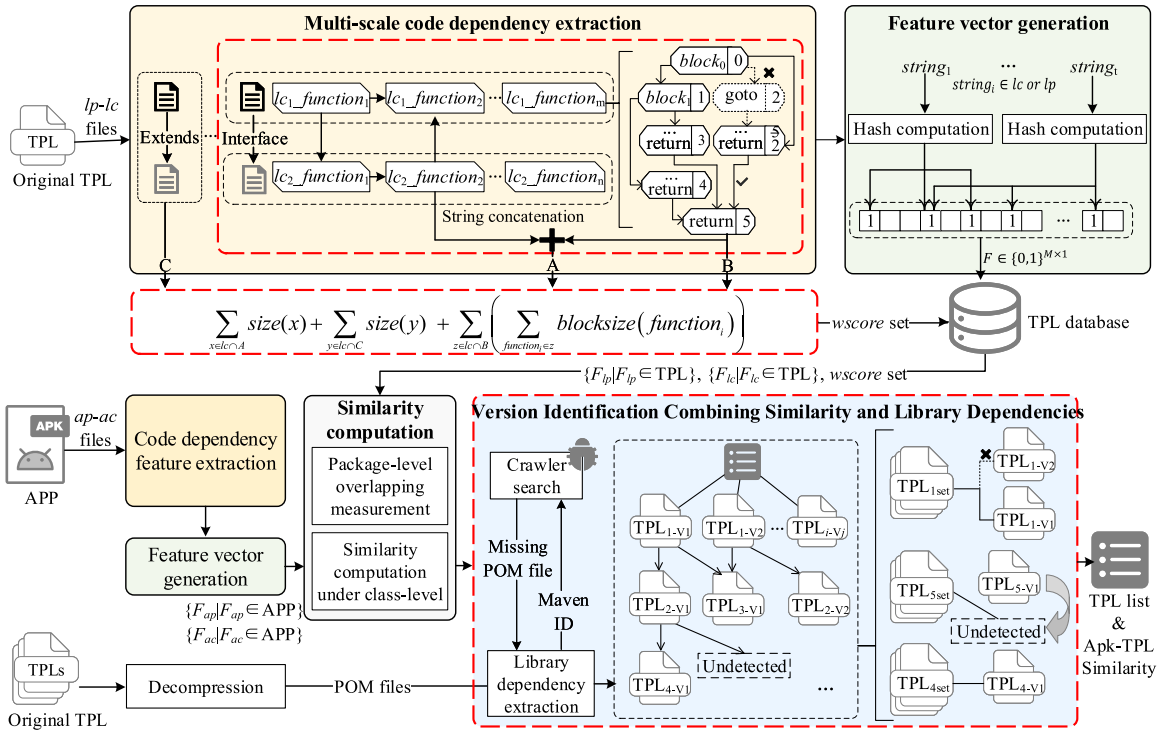


Fig. 1. The architecture of LibMD.

a significant number of functions, the app class of app may lack some opcodes, further diminishing the similarity between the TPL class and the app class. The subset query method calculates the similarity between TPLs and apps but struggles to accurately distinguish adjacent TPL versions when there is a large amount of identical code and only minor differences. For example, the detection results often include one correct version alongside multiple incorrect versions, which reduces the accuracy of TPL version identification.

3. Approach overview

3.1. Framework

Fig. 1 illustrates the framework of LibMD, which comprises four modules: multi-scale code dependency extraction, feature vector generation, similarity computation, and version identification that combines similarity and library dependency.

(1) Multi-scale code dependency extraction. This module is used to hierarchically extract code dependency features from apps or original TPLs. These features can be categorized into three scales: class, function, and basic block. Class-scale features (such as class inheritance and interfaces, fields, prototypes) are used for syntax enhancement based on modifiers and declaration formats. Function-scale features (such as intra-class and inter-class invocations) can integrate classes and basic blocks to obtain cross-scale function representations. Basic block-scale features can incorporate contextual instruction semantics to mitigate the impact of control flow randomization obfuscation and variations among different compilers on the CFG.

(2) Feature vector generation. This module generates package-level and class-level feature vectors based on Bloom filters [44] and calculates class weights according to the number of dependencies within each class. The code dependency features of the original TPL are used to create these feature vectors and class weights, which are then stored in a database (TPL database).

(3) Similarity computation. This module calculates the feature similarity between the TPL and the target app. First, candidate matching packages $candPkgs$ is selected based on the overlap rate of package-level feature vectors between the app and the TPL. Then, $candPkgs$ is filtered using differences in class-level feature vectors to identify the final matching packages $finPkgs$. The similarity between the TPL and the app is computed based on the number of classes in the $finPkgs$. A threshold-based decision is then used to identify TPLs within the app and form a candidate TPL list $candList$.

(4) Version identification combining similarity and library dependencies. First, an LDG is constructed based on the library dependencies of each TPL in the TPL database. Next, Bayes' theorem is applied to compute the posterior probability of each TPL and its adjacent versions being present in the app. Finally, the version with the highest posterior probability is retained, and any undetected TPLs are supplemented based on library dependency relationships.

3.2. Multi-scale code dependency extraction

In scenarios involving multiple types of code obfuscation, it is essential to integrate multi-scale code dependencies to mitigate the impact of changes in class dependencies or opcodes on critical library features. Therefore, based on obfuscation patterns, features related to class references, function invocations, and basic block control flows are extracted from the library code to maintain the usability of features after obfuscation. By combining multi-scale features, the interference caused by the absence of features at a single scale is reduced, thereby enhancing the obfuscation resistance of the detection method.

3.2.1. Dependency extraction of class references

Modifiers and declarations are utilized to enhance features of class references, accurately representing class dependencies in scenarios involving package flattening and class repackaging obfuscation. Class-scale code dependency features include the following three types:

(1) Inheritance and Interface. Inheritance and interfaces are foundational elements of the Java type system, and their dependency relationships typically remain unchanged under code obfuscation. The string "$edgeType$: $nodeType$" represents class inheritance or interface dependency relationships, where $edgeType$ denotes the type of

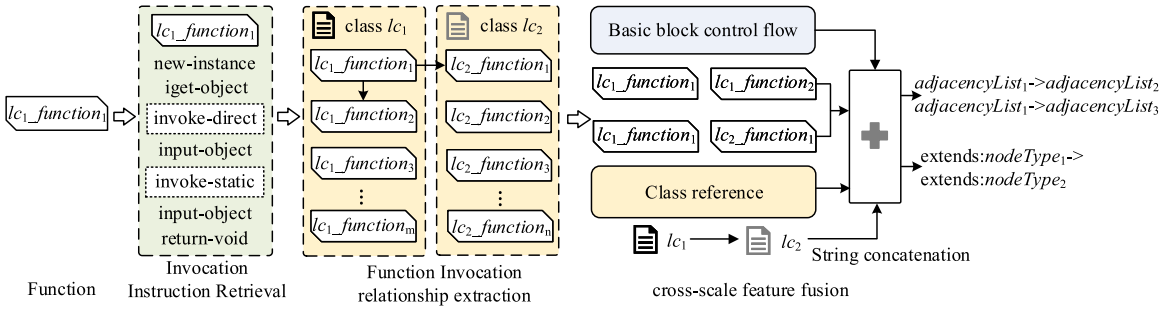


Fig. 2. The example of cross-scale function invocation.

Table 1
String mapping rules for data types.

Category	Data type	String
Basic system type	void	V
	boolean	Z
	int	I
	short	S
	long	L
	float	F
	double	D
	byte	B
	array	[
other system type	String	Ljava/lang/String
	Map	Ljava/util/Map
	List	Ljava/util/List
	default	Ljava/util/Object
app custom type	Custom type	X

relationship, typically using the keywords *extends* and *implements*, and *nodeType* represents the data type of the superclass. Additionally, the string "*abstract edgeType*" can represent dependencies involving abstract classes. To enhance the efficiency of character matching, the original data type is replaced with a character identifier, with the mapping relationship shown in Table 1. For example, in the class inheritance relationship "*abstract class a extends b*", if class *b* is a developer-defined data type, the class dependency can be expressed as "*abstract extends* : *X*".

(2) Field. A field is a member of a class. Field dependencies can be represented using a *nodeType* string. Adding the keywords *static* and *final* to a field can indicate whether it is a static field or a constant field. For example, the field dependency "*final int a* = 1" can be represented as "*final I*".

(3) Prototype. A prototype is a declaration of a function and is also a member of a class. Prototype dependencies can be expressed in the form of "*argumentType returnType*". For example, the prototype dependency "*static int A*(*double b, int c*)" can be expressed as "*static* (*D*; *I*)*I*".

3.2.2. Function invocation feature extraction

Fig. 2 shows the cross-scale invocation process of function $lc_1_function_1$: First, all invocation instructions in the function is searched, such as *invoke*λ*direct*, *invoke*λ*static*; Next, the function invocation relationships and invocation edges are extracted from these instructions, where an invocation edge is defined as a directed edge from the caller to the callee. Invocation edge includes two types: intra-class and inter-class invocations. For example, the intra-class invocation $lc_1_function_1 \rightarrow lc_1_function_2$ represents that the function invocation occurs within class lc_1, while the inter-class invocation $lc_1_function_1 \rightarrow lc_2_function_1$ indicates that the function invocation occurs between classes lc_1 and lc_2; Finally, function invocations are represented as a string "*caller* → *callee*", where both the *caller* and the *callee* are function entities, and the invocation relationship can be represented by the adjacency list of CFG. In the case of inter-class function invocations, an additional string "*caller class* → *callee class*" is used to represent the class inheritance relationship. For example, "*extends* : *X* → *extends* : *Ljava/util/Object*".

3.2.3. Basic block CFG reconstruction

The control flow of basic blocks is extracted to construct CFG. However, the CFG structure can be altered by unconditional jumps and return instructions. Therefore, during the CFG construction process, LibMD addresses this issue by removing single "goto" basic blocks and correcting multiple "return" basic blocks through context-aware instruction fusion. This ensures the stability of the CFG structure across different compilers and mitigates the effects of control flow randomization.

Algorithm 1 Semantics-based CFG reconstruction.

Input: The set of original basic blocks $\boldsymbol{V} = \{block_i \mid 1 \le i \le k\}$
Output: Adjacency list $\boldsymbol{S}$
1: Initialization: $FixExitBlockList = \emptyset$, $ExitBlock = null$
2: Construct CFG $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$ based on $\boldsymbol{V}$
3: **for** $block_i$ in $\boldsymbol{V}$ **do**
4: **if** $isSingleGoto(block_i)$ **then**
5: Remove $block_i$ from $\boldsymbol{V}$
6: Remove edges of $block_i$ from $\boldsymbol{E}$
7: Connect the predecessor node of $block_i$ to the successor node and add the edge to $\boldsymbol{E}$
8: **end if**
9: **if** $ExitBlock == null$ **and** $isSingleReturn(block_i)$ **then**
10: $ExitBlock = block_i$
11: **else if** $haveReturn(block_i)$ **then**
12: Add $block_i$ to $FixExitBlockList$
13: **end if**
14: **end for**
15: **if** $len(FixExitBlockList) \ge 1$ **then**
16: **if** $ExitBlock == null$ **then**
17: Construct a single return block $newExitBlock$
18: **for** $block_j \in FixExitBlockList$ **do**
19: Add the edge from $block_j$ to $newExitBlock$ to $\boldsymbol{E}$
20: **end for**
21: **end if**
22: **end if**
23: Extract adjacency list $\boldsymbol{S}$ from graph $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$
24: **return** $\boldsymbol{S}$

The function control flow reconstruction process is illustrated in Algorithm 1. First, given a set of basic blocks $\boldsymbol{V} = \{block_i \mid 1 \le i \le k\}$ within a function, its CFG $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$ is directly constructed, where $\boldsymbol{V}$ represents node (basic block) set and $\boldsymbol{E}$ represents the edges between nodes. In lines 4 to 8, each basic block $block_i$ containing only one unconditional jump instruction *goto* can be retrieved by traversing $\boldsymbol{V}$. In order to reduce the interference of instructions "*goto*" on CFG analysis, a new dependency is established by deleting the node $block_i$ and connecting its predecessor and successor nodes. Second, in lines 9 to 13, the set $\boldsymbol{V}$ is examined to locate a single "return basic block" (a

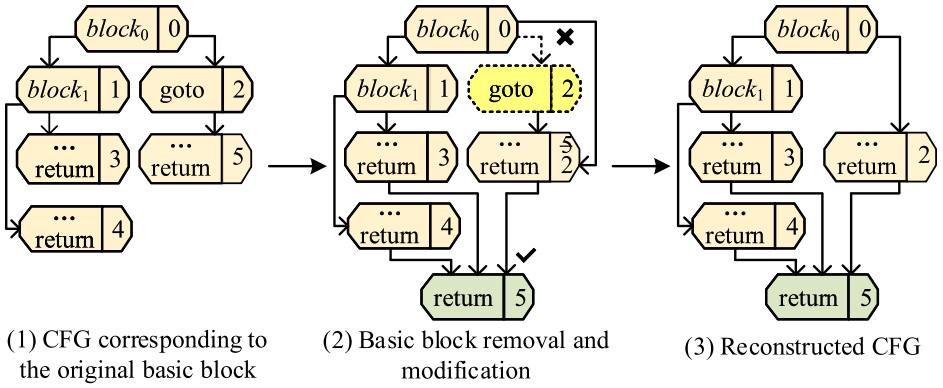


Fig. 3. Basic block CFG reconstruction.

block containing only one *return* instruction), referred to as $Exitblock$. Other return blocks are added to the set $FixExitBlockSet$. Lines 15 to 22 outline the handling of the $Exitblock$. If no $Exitblock$ is found in $\boldsymbol{V}$, a new $Exitblock$ is constructed. Each basic block in $FixExitBlockSet$ is then provided with an outgoing edge pointing to the $Exitblock$, ensuring that the CFG has only one exit block. Finally, an adjacency list for the graph $\boldsymbol{G}$ is generated. Each node is assigned a unique number in ascending order, starting from the entry block. When multiple successor nodes are present, those with a higher out-degree are prioritized for numbering. If successor nodes have the same out-degree, those with a greater number of instructions are numbered first. If the number of instructions is also the same, successors with smaller storage addresses are given priority.

An example of basic block CFG reconstruction is shown in Fig. 3. Fig. 3(a) shows a CFG constructed using the original basic block set, which contains one "*goto*" basic block (numbered 2) and multiple "*return*" basic blocks (numbered 3, 4, and 5). The corresponding adjacency list string is "$0 \rightarrow (1,2), 1 \rightarrow (3,4), 2 \rightarrow (5), 3 \rightarrow (), 4 \rightarrow (), 5 \rightarrow ()$". After CFG reconstruction using Algorithm 1, the single "*goto*" block is removed and multiple "*return*" blocks are connected to an exit block. As shown in Fig. 3(b), the final adjacency list string corresponding to the CFG is "$0 \rightarrow (1,2), 1 \rightarrow (3,4), 2 \rightarrow (5), 3 \rightarrow (5), 4 \rightarrow (5), 5 \rightarrow ()$". Different compilers and control flow randomization obfuscation techniques often insert a single "*goto*" basic block and redundant "*return*" basic blocks (or instructions) to change the CFG structure. LibMD can maintain the stability and integrity of CFG features by fusing the context instruction semantics.

3.2.4. Feature vector generation

The feature vector generation module uses unbiased hash encoding to embed code dependency features, reducing the high time complexity of string matching. The module generates feature vectors for each package and class in TPLs using Bloom filters. The hash functions are constructed using MurmurHash3 [45] and double hashing techniques. To mitigate false positives due to Bloom filter limitations and improve detection performance, we have set appropriate parameters, as detailed in Section 4.2.

To improve the accuracy of similarity computations, the class weight for each class-level feature vector is computed based on LIBLOOM [28], as shown in Eq. (1).

$$\begin{aligned} Wscore(lc) = &\sum_{x \in lc \cap A} size(x) + \sum_{y \in lc \cap C} size(y) \\ &+ \sum_{z \in lc \cap B} \left(\sum_{function_i \in z} blocksize(function_i) \right) \end{aligned} \tag{1}$$

where lc represents classes in the TPL, and $Wscore(lc)$ denotes the weight of lc. The feature sets A, B, and C correspond to function invocation, semantic CFG, and class reference, respectively. $function_i$ represents a function in TPL, $blocksize(\cdot)$ denotes the number of basic blocks in a function.

3.3. Similarity computation

The module is used to calculate the feature similarity between each TPL in the TPL database and the target app. The similarity computation is divided into two stages:

In the first stage, the feature vector overlap ratio between the app package and the TPL package is calculated, as shown in Eq. (2). If the value of $overlap_ratio(lp, ap)$ exceeds the threshold, the second stage is performed to identify a candidate package set $candPkgs$ that includes both packages lp and ap.

$$overlap_ratio(lp, ap) = \frac{F_{lp} \cdot F_{ap}}{min(\left|F_{lp}\right|, \left|F_{ap}\right|)} \tag{2}$$

$$\begin{aligned} candPkgs = &\{(lp_x, ap_y) \mid lp_x \in lp, ap_y \in ap, \\ &overlap_ratio(lp_x, ap_y) \geq 0.8\} \end{aligned} \tag{3}$$

where lp denotes a package from the TPL, and ap represents a package from the app. $F_{lp} \cdot F_{ap}$ is the dot product of two package feature vectors. $|F|$ denotes the number of element equal to "1" in the vector F. $min(a, b)$ refers to the smaller value between a and b. $overlap_ratio(lp, ap)$ denotes the overlap rate of the package feature vectors of lp and ap.

In the second stage, the similarity between the TPL class lc and the app class ac is calculated, as shown in Eq. (4).

$$sim_class(lc_i, ac_j) = \frac{F_{lc_i} \cdot F_{ac_j}}{\left|F_{lc_i}\right|} \tag{4}$$

The similarity between the TPL package lp and the app package ap is shown in Eq. (5). Since each TPL package or app package consists of multiple classes, matching different classes will yield different package similarities. Therefore, the Kuhn-Munkres algorithm is used to determine the optimal class matching result.

$$\begin{aligned} &sim_package(lp_x, ap_y) = \\ &\quad KM\left(\sum_{lc_i \in lp_x, ac_j \in ap_y} \frac{sim_class(lc_i, ac_j) * Wscore(lc_i)}{\sum_{lc_k \in lp_x} Wscore(lc_k)} \right) \\ &\quad , (lp_x, ap_y) \in candPkgs \end{aligned} \tag{5}$$

where $sim_package(lp_x, ap_y)$ represents the similarity between TPL package lp_x and app package ap_y, $KM(\cdot)$ denotes the optimal class matching result obtained using the Kuhn-Munkres algorithm.

Then, for each TPL package lp in the candidate package set $candPkgs$, it is necessary to find the app package ap that maximizes the package similarity and obtain the final matching package set $finPkgs$, as shown in Eq. (6).

$$finPkgs = \underset{lp_x \in candPkgs,\ ap_z \in ap}{argmax} sim_package(lp_x, ap_z) \tag{6}$$

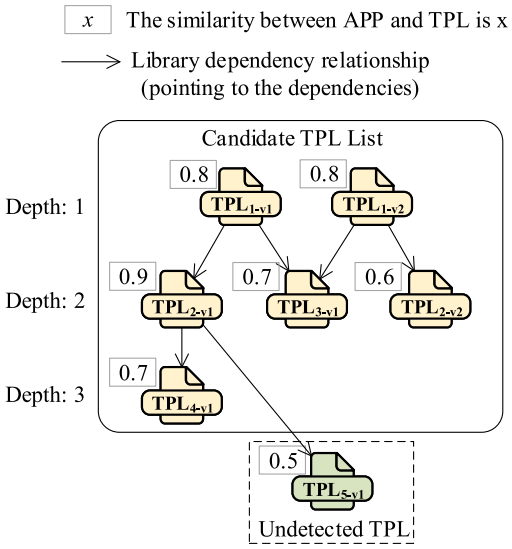


Fig. 4. Example of LDG and posterior probability calculation.

Finally, the similarity between the app and the TPL is calculated based on the number of classes in all matching packages in $finPkgs$, as shown in Eq. (7).

$$similarity(tpl, app) = \left(\frac{\sum_{ap_j \in app} countClasses(ap_j)}{\sum_{lp_i \in tpl} countClasses(lp_i)} \right), (lp_i, ap_j) \in finPkgs \tag{7}$$

where $countClasses(\cdot)$ represents the number of classes contained in either the TPL or the app package. If the similarity $similarity(lp, ap)$ between the app and the TPL satisfies or exceeds a predefined threshold, the TPL and its corresponding similarity value are added to the candidate TPL list.

3.4. Version identification combining similarity and library dependencies

The code similarity between different versions of a TPL and a given app is often compared to identify the correct library version. However, adjacent versions of a TPL tend to be closely clustered in the feature space, resulting in similar or nearly identical similarity scores, which reduces the accuracy of identifying the correct version. One approach is to leverage the subtle differences between multiple TPLs with dependencies to enhance the distinction between adjacent versions, thereby improving the accuracy of version identification.

This module analyzes library dependencies within TPL metadata [46] and combines code similarities across multiple TPLs and apps to construct a LDG. Given the known similarities, Bayes' theorem is used to calculate the posterior probability of a specific version of a TPL being present in the app. The module then selects the most likely version from the candidate TPL list $candList$ based on this posterior probability to determine the final TPL list $finList$. An example of the LDG and posterior probability calculation is illustrated in Fig. 4.

3.4.1. Library dependency mining in TPL metadata

Mining library dependencies from TPL metadata involves analyzing various pieces of information that describe and manage external dependencies, which are typically found in POM files. These POM files contain structured metadata, such as dependency lists, version numbers, and hierarchical relationships, that are essential for constructing a LDG. The module first searches for POM files within the original TPL

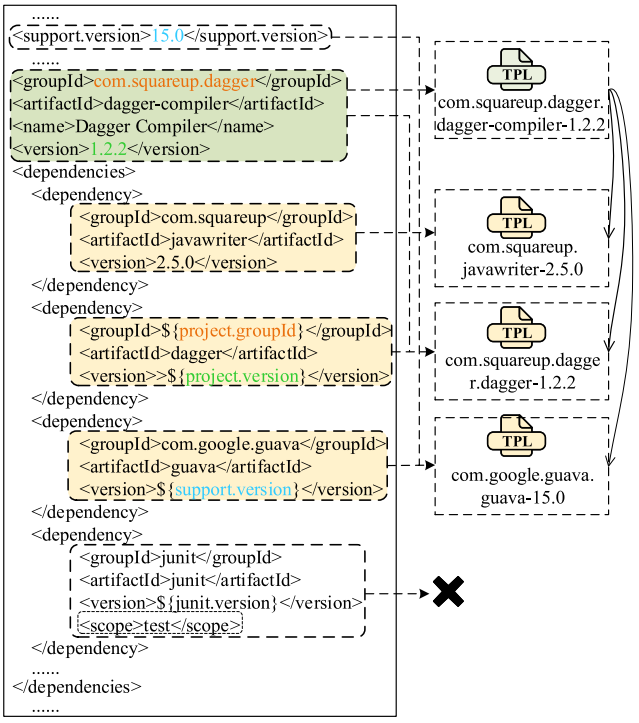


Fig. 5. The extraction library dependencies.

packages. If a POM file is missing, a custom crawler is deployed to search online repositories, such as Maven Repository, to retrieve the required metadata. Once collected, the module extracts all dependency information from the POM files, including transitive dependencies, and integrates them into a comprehensive LDG within the TPL database. This process ensures a complete and accurate representation of library relationships, which is critical for precise TPL detection and version identification.

Fig. 5 illustrates the POM file and library dependencies for the TPL "$dagger - compiler - 1.2.2$". The process begins by locating the tag $\langle dependencies \rangle$ and extracting the Maven unique identifier tuple, which includes the $groupId$, $artifactId$, and $version$. Next, these tuple variables are resolved to extract the TPL dependencies. For tuples without variable substitution, dependencies can be directly extracted, such as "$com.squareup.javawriter - 2.5.0$". In cases where variables are used, if the variable format is "$\$project.x$", it indicates that the dependency tuple information is consistent with the current TPL, such as "$com.squareup.dagger : dagger - 1.2.2$". For variables in the format of "$\$support.x$" or "$\$junit.x$", it is necessary to traverse the POM file to find the corresponding values. For example, the version for the tuple corresponding to "$com.google.guava.guava - 15.0$" can be determined from the $\langle support.version \rangle$ in the POM file.

3.4.2. Candidate TPL update

The update process for candidate TPLs is described in Algorithm 2. The maximum depth, $depth_{max}$, of the LDG is determined to compute the depth-weighted set. In the LDG, child nodes closer to the root have a tighter dependency relationship with the TPL, resulting in higher depth weights. The calculation of these weights is given by Eq. (8):

$$Hweight_x = \frac{depth_{max} - x + 1}{\sum_{y \in \{z | z \le depth_{max}, z \in N^+\}} (depth_{max} - y + 1)} \tag{8}$$

where $Hweight_x$ represents the weight of depth x, $depth_{max}$ is the max depth of LDG. For example, in Fig. 4, if the maximum depth of the constructed LDG is 3, the weights corresponding to depth 1, depth 2, and depth 3 are 1/2, 1/3, and 1/6, respectively.

Furthermore, for TPL tpl_i with multiple versions, the probability of each version $tpl_{i,j} \in tpl_i$ being present in the app can be calculated using Eq. (9).

$$Pv_j = P(Y_j \mid X_j) = \frac{P(X_j \mid Y_j) \cdot P(Y_j)}{P(X_j)} \tag{9}$$

where event Y_j denotes the presence of $tpl_{i,j}$ in the app, event X_j indicates the detection of a specific similarity. $P(Y_j)$ denotes the prior probability, representing the initial belief that the app contains $tpl_{i,j}$, and can be set to 0.5. $P(X_j \mid Y_j)$ denotes the likelihood, representing the probability of detecting a specific similarity given that the app contains $tpl_{i,j}$. $P(X_j)$ denotes the marginal probability, representing the overall probability of detecting a specific similarity. $P(Y_j \mid X_j)$ denotes the posterior probability, representing the probability that the app contains $tpl_{i,j}$ given the specific similarity.

Since $tpl_{i,j}$ has library dependencies with other TPLs, the likelihood $P(X_j \mid Y_j)$ can be calculated using Eq. (10).

$$P(X_j \mid Y_j) = \sum_{tpl_{p,q} \in subLDG} \frac{Hweight_q}{N_q} \times similarity\left(tpl_{p,q}, app\right) \tag{10}$$

where $subLDG$ represents the library dependency subgraph with $tpl_{i,j}$ as the root node. N_q denotes the number of nodes at the same depth within the scope of the $subLDG$. $tpl_{p,q}$ represents any node within the $subLDG$, $Hweight_q$ indicates the weight of the node at depth q, and $similarity\left(tpl_{p,q}, app\right)$ denotes the similarity between $tpl_{p,q}$ and app as calculated by Eq. (7).

In Fig. 4, the subgraph $subLDG$ with root node $tpl_{1,1}$ includes $tpl_{1,1}$, $tpl_{2,1}$, $tpl_{3,1}$, and $tpl_{4,1}$ and their dependent edges (excluding $tpl_{5,1}$ because it does not exist in the candidate TPL list). Since the subgraph at depth 2 includes two nodes, $tpl_{2,1}$ and $tpl_{3,1}$, under the condition that the app contains $tpl_{1,1}$, the likelihood of observing similarity X_1 is approximately $P(X_1 \mid Y_1) \approx 0.93$. Similarly, $P(X_2 \mid Y_2) \approx 0.72$.

3.4.3. Final TPL list construction

The version corresponding to the maximum posterior probability is added to the final TPL list $finList$. Versions that belong to the candidate TPL list $candList$ but are part of the $subLDG$ are also included in the $finList$. For example, in Fig. 4, the posterior probabilities of $tpl_{1,1}$ and $tpl_{1,2}$ in the app are $P(Y_1 \mid X_1) \approx 0.58$ and $P(Y_2 \mid X_2) \approx 0.45$, respectively. Therefore, among the multiple versions of tpl_1 in the $candList$, version $tpl_{1,1}$ is added to the $finList$, and version $tpl_{1,5}$ from the $subLDG$ (with root node $tpl_{1,1}$) is also added to the $finList$.

Algorithm 2 Candidate TPL list update.

Input: Candidate TPL list $candList$, Library dependency graph LDG
Output: Final TPL list $finList$

```
Initialization: finList = []
Obtain the maximum depth depth_max of LDG
Calculate the depth weight set Hweight using Equation (8)
for tpl_i ⊂ candList do
    Pv is initialized as a zero array of length len(tpl_i)
    for tpl_{i,j} ∈ tpl_i do
        Calculate the likelihood tpl_{i,j} using Equation (10)
        Calculate the Pv_j of tpl_{i,j} using Equation (9)
    end for
    Obtain max(Pv_j) and its corresponding library tpl_{i,max}
    Obtain subLDG with root node tpl_{i,max}
    Add all TPLs in the subLDG to finList
end for
return finList
```

3.5. Time complexity analysis

Given the computational demands and privacy protection requirements, it is essential to evaluate the complexity of TPL detection tools to ensure their practicality and reliability [47,48]. Among the various processes, the feature comparison strategy plays a pivotal role in determining the efficiency of TPL detection tools. Given a test app, where the number of packages is denoted as p, the number of classes as c in an app package, and the number of methods as m in an app class, it is generally the case that $m > c > p$ [20]. Let n represent the total number of TPLs stored in the feature database. We use $\bar{p}$ to denote the number of packages in a TPL that may match, $\bar{c}$ to represent the number of classes in a package, and $\bar{m}$ to represent the number of methods in a class.

For LibPecker, it performs similarity comparisons at both the package level and the class level, with optimal matching applied to classes. Therefore, its complexity is $O(p{\times}n{\times}\bar{p}){+}O(p{\times}c{\times}p{\times}\bar{c}){+}O((p{\times}c)!)$. LibRoad employs package name-based matching for non-obfuscated packages and signature-based matching for obfuscated packages in the app. As a result, its time complexity is $O(p_1 \times n \times \bar{p}) + O(p_2 \times c \times n \times \bar{p} \times \bar{c})$, where p_1 is the number of non-obfuscated packages in the app, and p_2 is the number of obfuscated packages in the app. LibScan sequentially applies class-based, method-based, and Call-Chain-based matching. Therefore, its time complexity is $O(p \times c \times n \times \bar{p} \times \bar{c}) + O(p \times c_o \times n \times \bar{p} \times \bar{c_o} \times m \times \bar{m}) + O(p \times c_c \times n \times \bar{p} \times \bar{c_c} \times d \times k)$, where c, c_o, and c_c represent the number of classes, with $c \geq c_o \geq c_c$, d represents the depth of the call chain, with each method calling k methods. In the feature matching phase, LibMD and LIBLOOM have the same time complexity because they adopt the same strategy.

The feature comparison strategy of LibMD consists of three phases: package-level matching, class-level matching, and matching optimization. In the package-level matching phase, the complexity of package-level similarity computation is $O(p \times n \times \bar{p})$. In the class-level matching phase, for candidate packages, the similarity between classes is calculated, and this phase has a complexity of $O(c \times p_a \times \bar{c})$, where p_a is the number of candidate packages. In the matching optimization phase, the Kuhn-Munkres algorithm is used for optimal class matching, with a complexity of $O(max(p_a{\times}c, p_a{\times}\bar{c})^3)$. Combining these phases, the overall time complexity of LibMD is $O(p{\times}n{\times}\bar{p}){+}O(c{\times}p_a{\times}\bar{c}){+}O(max(p_a{\times}c, p_a{\times}\bar{c})^3)$. After analysis, we conclude that LIBLOOM demonstrates superior efficiency in terms of time complexity compared to other TPL detection methods.

4. Experiments and analysis

In this section, the performance of LibMD is evaluated in the following three aspects.

(1) Threshold experiment: To validate the effectiveness of the preset thresholds, the detection results of TPLs in LibMD are compared across different similarity thresholds using the open-source dataset LibDetect.

(2) Comparative experiment: LibMD is compared with four state-of-the-art Android TPL detection methods on three open-source datasets to validate its effectiveness.

(3) Ablation experiment: By incrementally removing the code dependency extraction module (CD) and the version identification module (LD), the impact of these two modules on the performance of LibMD is analyzed.

4.1. Data source and set up

The dataset for detecting TPLs in Android apps consists of three main types: (1) Non-Obfuscated Dataset: This dataset features minimal differences between adjacent TPL versions and is used to validate the version identification capability of method; (2) Single-Type Obfuscation Dataset: This dataset includes a single type of code obfuscation and is typically used to assess the adaptability of method to a single obfuscation type; (3) Composite-Type Obfuscation Dataset: This dataset includes two types of code obfuscation and is commonly used to evaluate the practical applicability of the method.

Table 2
Experimental dataset.

Id	Dataset	Apps	App obfuscation type	TPLs	TPLs after deduplication
1	Wang [18]	62	none	781	451
2	Zhan [43]	88*3	Fltn, Ctrl, Rmv	487	451
3	Huang [28]	100*8	none, PGD, PGR, PGF, ALW, ALS, DOR, DOF	938	551
4	LibDetect [9]	162*3	ProGuard, Allatori, DashO	781	451

Table 3
Enabled obfuscation strategies of each obfuscator.

Dataset		Obfuscator	Description			
			DCR	CFR	PF	CR
Zhan	Fltn	DashO			✓	
	Ctrl	DashO		✓		
	Rmv	DashO	✓			
Huang	PGD	ProGuard	✓			
	PGR	ProGuard		✓		✓
	PGF	ProGuard	✓		✓	
	ALW	Allatori	✓			✓
	ALS	Allatori	✓			✓
	DOR	DashO	✓			✓
	DOF	DashO	✓		✓	

DCR, CFR, PF and CR represent four types of code obfuscation: dead code removal, control flow randomization, package flattening, and class repackaging, respectively.

Table 4
Comparative experiment results on the dataset without code obfuscation.

Dataset		LibPecker	LibRoad	LIBLOOM	LibScan	LibMD
Wang	Precision	58.35%	88.78%	76.29%	92.46%	**93.23%**
	Recall	97.06%	87.20%	98.46%	97.95%	**99.10%**
	F1	72.88%	87.98%	85.97%	95.13%	**96.08%**
Huang	Precision	28.75%	52.12%	70.14%	30.06%	**74.42%**
	Recall	**95.63%**	92.00%	93.03%	11.41%	93.71%
	F1	44.21%	66.54%	80.35%	16.54%	**82.96%**
Average	Precision	43.55%	70.45%	73.22%	61.26%	**83.83%**
	Recall	96.35%	89.60%	95.75%	54.68%	**96.41%**
	F1	58.55%	77.26%	82.97%	55.83%	**89.52%**

The non-obfuscated part of the Huang dataset is used for testing.

Four publicly available datasets in the fields of Android TPL detection and version identification are used for performance evaluation, with their basic information summarized in Table 2. These datasets are sourced from authentic and widely recognized platforms, including Google Play, the F-Droid repository, Maven, and the Xiaomi App Store, providing legitimate APK files and comprehensive metadata to ensure their authenticity and reliability. The datasets from Wang [18], Zhan [43], and Huang [28] are employed for comparative experiments to evaluate the version identification and obfuscation resistance of each method, with their code obfuscation settings detailed in Table 3. The composite-type code obfuscation dataset, LibDetect, is used for threshold experiments to validate the reasonableness and general applicability of the similarity thresholds.

These datasets, widely recognized in prior research, ensure that the evaluation is aligned with state-of-the-art methodologies and firmly grounded in established practices. They consist of obfuscated apps processed using prominent obfuscation tools such as ProGuard for code shrinking, Allatori for repackaging, and DashO for control flow flattening. This selection effectively reflects real-world obfuscation techniques and provides a robust foundation for assessing obfuscation resistance. While the datasets offer extensive coverage of common obfuscation methods, they may have limitations in representing less prevalent or emerging techniques. Additionally, the inclusion of apps from specific sources, such as the Xiaomi App Store, could introduce geographic bias. However, the combination of datasets drawn from diverse origins, including open-source apps, closed-source apps, and large-scale benchmarks, ensures a well-rounded and comprehensive representation of real-world Android development and obfuscation scenarios.

4.2. Experimental environment

We use a Windows 11 operating system on a machine with an Intel Core i9-9900 CPU @ 3.10 GHz and 32 GB of RAM. The implementation is developed in Java 8.

Based on the settings of LibLOOM [28], we directly adopt its Bloom filter parameters. Specifically, the package-level feature vector length M_p is set to 21,640 with $k_p = 3$ hash functions, and the class-level feature vector length M_c is set to 256 with $k_c = 3$ hash functions. These parameters are carefully optimized in LibLOOM [28] to achieve a false positive rate below 0.01 while maintaining high detection accuracy and computational efficiency. This configuration ensures scalability for large-scale library detection and avoids false negatives, guaranteeing precise identification of valid matches. By leveraging these well-tested parameters, our implementation achieves reliable performance without additional parameter tuning.

4.3. Evaluation method

Three indicators are used in the experimental process, namely $Precision$, $Recall$, and $F1$.

$$Precision = \frac{TP}{TP + FP} \tag{11}$$

$$Recall = \frac{TP}{TP + FN} \tag{12}$$

$$F1 = \frac{2 * precision * recall}{precision + recall} \tag{13}$$

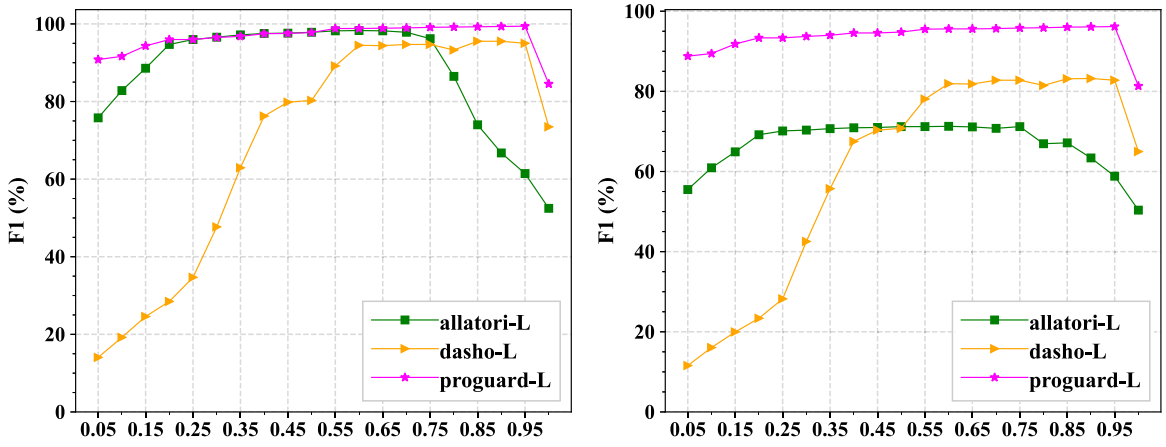
where TP denotes the number of TPLs correctly detected in the app; FP denotes the number of TPLs incorrectly identified as present in the app when they are not; and FN denotes the number of TPLs incorrectly identified as absent in the app when they are present.

4.4. Threshold experiment

To find a reasonable similarity threshold, the library-level and version-level detection results of LibMD are separately evaluated on the LibDetect dataset. The F1 scores for library detection and version identification, as well as the average F1 scores, are shown in Fig. 6 and Fig. 7, respectively.

Fig. 6 illustrates the variation in F1 scores across different similarity thresholds. Lower thresholds lead to a reduction in F1 scores due to the inclusion of a larger number of candidate libraries, which increases the likelihood of false positives. Moreover, as discussed in Section 3.5 regarding the time complexity of different methods, the inclusion of more candidates results in higher computational costs. Conversely, higher thresholds reduce computational time by limiting the number of comparisons, but this comes at the cost of missing certain libraries, particularly those with subtle feature similarities. These results highlight the importance of selecting an appropriate similarity threshold to balance detection performance and computational efficiency.

LibMD achieves better detection performance with a similarity threshold of 0.6. As the threshold increases, both library and version-level detection F1 scores initially rise and then decline, with the average F1 score peaking in the range of 0.6 to 0.7. Given that real-world scenarios often involve higher complexity, such as increased



(a) F1 score for library detection

(b) F1 score for version identification

Fig. 6. Threshold experiment results on the LibDetect dataset.

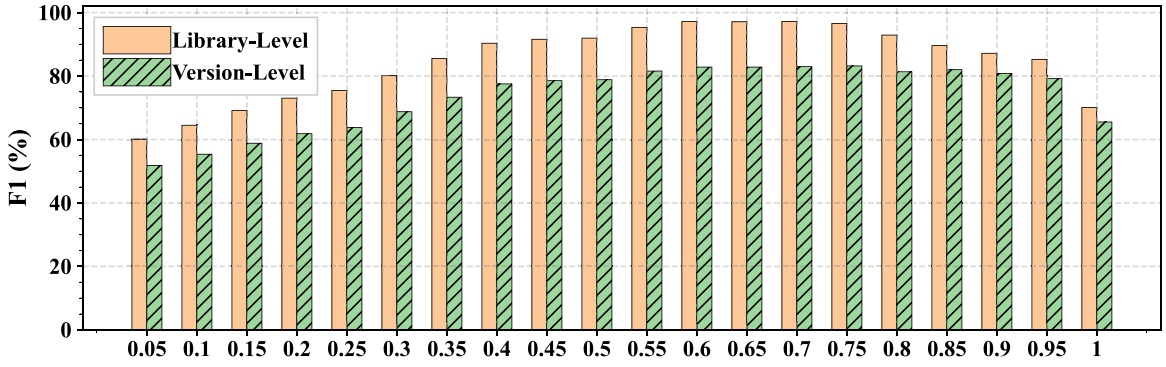


Fig. 7. The experimental results under various similarity thresholds.

obfuscation or variations in code features due to different compilation environments, setting the similarity threshold to 0.6 is necessary to maintain the stability of the detection performance LibMD.

4.5. Comparison experiment

The performance of LibMD with four TPL detection methods (LibPecker [22], LibRoad [19], LIBLOOM [28], and LibScan [17]) on three datasets (Wang [18], Zhan [43], and Huang [28]) is compared. The effectiveness of LibMD is verified by calculating *Precision*, *Recall*, and $F1$ score. Each method requires setting similarity thresholds between app and TPL, with LibMD using a threshold of 0.6, while the other methods employ the optimal thresholds reported in their respective papers. Additionally, independent repeated experiments with varying similarity thresholds are conducted to investigate the impact of similarity on TPL detection performance. The similarity threshold range is set from 0.05 to 1.00, with a step size of 0.05.

The detection results of LibMD and other methods on datasets with and without code obfuscation are shown in Tables 4 and Table 5, respectively. Results under different similarity thresholds are illustrated in Fig. 8. The results of detecting vulnerable TPLs in the unobfuscated portion of the Huang dataset are presented in Table 6. The comprehensive analysis of the experimental results is as follows:

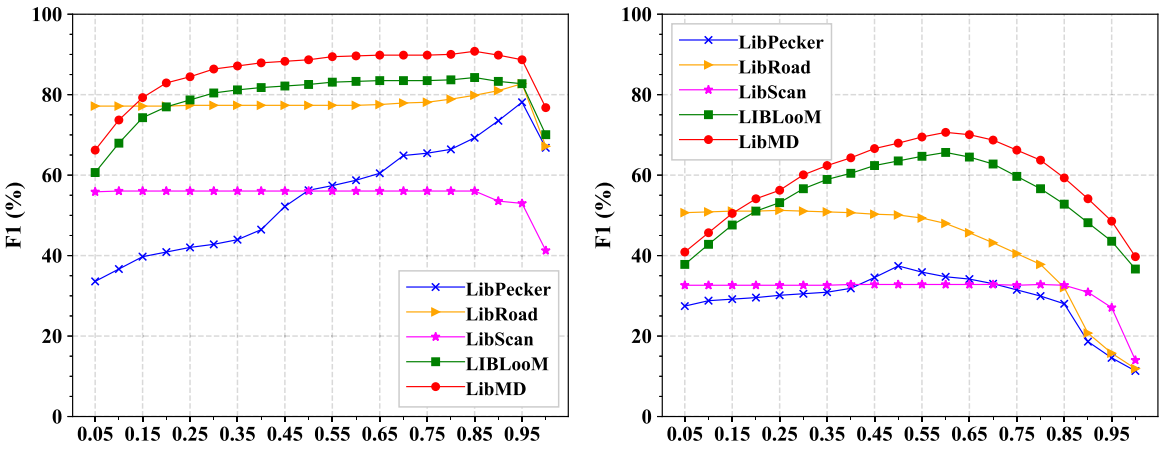
4.5.1. Performance comparison with code obfuscation

In the absence of code obfuscation, LibMD demonstrates superior performance in TPL version identification compared to the other methods.

(1) In the non-obfuscated code dataset (including the Wang and non-obfuscated part of Huang), compared to state-of-the-art methods, LibMD achieves an average precision increase of 10.61% and an F1 score increase of 6.36%, with recall remaining nearly the same (increased by 0.06%). When the similarity threshold is at least 0.15, the F1 score of LibMD consistently exceeds that of other methods. Although the F1 score of LibMD is lower than some methods when the threshold is below 0.15, a lower threshold leads to false positives, which have no practical significance for the detection task.

(2) Since opcode features contain specific instruction information, LibScan exhibits higher detection precision than other methods on the Wang dataset. However, the opcode in Smali is strongly influenced by the app compilation environment and decompilation tools. Consequently, instructions with the same functionality, such as “mul-float/2addr a, b” and “mul-float a, a, b” (both performing floating-point multiplication but with different opcodes), may result in missed matches between apps and TPLs. This discrepancy reduces the precision and recall of LibScan detection on the Huang dataset (unobfuscated portion). In contrast, LibMD avoids direct opcode matching by extracting small-scale contextual instruction semantics and CFG features, thereby achieving accurate version identification across multiple datasets.

(3) LibPecker relies solely on class dependency features for TPL detection. As class dependency features remain unchanged despite internal code variations within functions during the app compilation process, LibPecker achieves higher recall than LibMD on the Huang dataset (unobfuscated portion). However, class dependency features cannot represent code dependencies or instruction semantics at the function level, making it challenging to distinguish between functions with similar structures and internal code differences. Consequently, LibPecker exhibits lower precision. In contrast, LibMD incorporates code dependency features at the class, function, and basic block scales,



(a) dataset without code obfuscation (b) dataset with code obfuscation

Fig. 8. The average F1 score of the comparative experiment.

Table 5
Comparative experiment results on the dataset with code obfuscation.

Dataset	Obfuscation type		LibPecker	LibRoad	LIBLOOM	LibScan	LibMD
Zhan	Fltn	Precision	54.61%	85.05%	77.96%	96.38%	**96.94%**
		Recall	64.48%	64.48%	89.32%	97.33%	**97.54%**
		F1	59.14%	73.35%	83.25%	96.85%	**97.24%**
	Ctrl	Precision	56.59%	84.72%	78.85%	95.75%	**98.16%**
		Recall	72.28%	71.05%	98.77%	96.92%	**98.77%**
		F1	63.48%	77.29%	87.69%	96.33%	**98.46%**
	Rmv	Precision	54.70%	85.05%	77.55%	**97.08%**	92.24%
		Recall	64.48%	64.48%	93.63%	86.24%	**94.66%**
		F1	59.19%	73.35%	84.83%	91.34%	**93.43%**
Huang	PGD	Precision	35.26%	40.02%	75.95%	19.10%	**79.49%**
		Recall	27.40%	36.25%	47.12%	1.81%	**50.32%**
		F1	30.84%	38.04%	58.16%	3.31%	**61.63%**
	PGR	Precision	34.13%	60.43%	64.45%	18.82%	**66.21%**
		Recall	18.44%	24.41%	36.14%	1.71%	**40.94%**
		F1	23.94%	34.77%	46.31%	3.14%	**50.60%**
	PGF	Precision	36.61%	58.84%	68.64%	18.82%	**70.63%**
		Recall	19.94%	25.48%	45.84%	1.71%	**49.15%**
		F1	25.82%	35.56%	54.97%	3.14%	**57.96%**
	ALW	Precision	30.19%	49.22%	67.11%	28.12%	**67.64%**
		Recall	57.89%	71.00%	80.92%	10.34%	**81.02%**
		F1	39.68%	58.14%	73.37%	15.12%	**73.73%**
	ALS	Precision	15.64%	45.29%	54.21%	15.79%	**55.34%**
		Recall	3.52%	33.26%	86.35%	4.80%	**86.57%**
		F1	5.75%	38.35%	66.61%	7.36%	**67.52%**
	DOR	Precision	36.99%	59.30%	62.02%	30.30%	**64.03%**
		Recall	13.65%	16.31%	36.03%	3.20%	**40.30%**
		F1	19.94%	25.58%	45.58%	5.79%	**49.47%**
	DOF	Precision	39.40%	59.53%	69.38%	31.00%	**69.62%**
		Recall	14.07%	16.31%	47.33%	3.30%	**49.36%**
		F1	20.74%	25.60%	56.27%	5.97%	**57.77%**
Average		Precision	39.41%	62.75%	69.61%	45.12%	**76.03%**
		Recall	35.62%	42.30%	66.15%	30.74%	**68.86%**
		F1	34.85%	48.00%	65.70%	32.83%	**70.78%**

The obfuscated part of the Huang dataset is used for testing.

Table 6
Detection results of vulnerable TPLs on the Huang dataset (non-obfuscated portion).

Number	Lib-Pecker	Lib-Road	LIB-LOOM	Lib-Scan	LibMD
TP	67	66	66	0	**71**
FP	405	71	24	**1**	4
TP (total)	71	71	71	71	71

balancing both recall and precision, and thus demonstrates superior overall performance in version identification.

(4) Compared to other methods, LibMD accurately identifies vulnerable TPLs, revealing potential security risks and threats in apps. On the Huang dataset (unobfuscated portion), LibMD can detect TPLs with known vulnerabilities, including: CVE-2022-25647 in TPL "gson" (pre-version 2.8.9), CVE-2022-36033 in TPL "jsoup" (pre-version 1.15.3), CVE-2018-1000850 and CVE-2018-1000844 in TPL "retrofit" (pre-version 2.5.0), and CVE-2022-25845 in TPL "fastjson" (pre-version 1.2.83). In contrast, the opcode features of LibScan are strongly tied

to the app compilation environment and decompilation tools, making TPLs in the Huang dataset difficult to detect. As a result, while LibScan has fewer false positives in detecting vulnerable TPLs, its practical utility is limited due to a zero true positive rate. LibMD, on the other hand, successfully identifies a greater number of vulnerable TPLs while producing fewer false positives.

4.5.2. Performance comparison without code obfuscation

In the presence of code obfuscation, the anti-obfuscation ability of LibMD is better than the comparison method.

(1) In the code-obfuscated datasets Zhan and Huang (which contain obfuscated sections), LibMD achieves average improvements of 6.42% in precision, 2.72% in recall, and 5.08% in F1 score compared to other state-of-the-art methods. When the similarity threshold is set to 0.2 or higher, LibMD consistently outperforms other methods in terms of F1 score. However, while the F1 score of LibMD is slightly lower than that of some methods when the threshold is below 0.2, excessively low thresholds result in a higher rate of false positives, which are not meaningful for detection tasks.

(2) LibScan achieves a higher F1 score on the Zhan dataset (Fltn, Ctrl, Rmv) but shows a significant decline on the Huang dataset. In addition to the reasons mentioned in (1), this drop is primarily attributed to the weaker resistance of LibScan to dead code removal obfuscation. For example, compared to PGD, PGR, PGF, DOR, and DOF, LibScan demonstrates a higher F1 score on ALW and ALS obfuscation types. This is because, for the same app, the extent of dead code removal performed by obfuscators ProGuard and DashO is greater than that by Allatori. In contrast, across all obfuscation types in both datasets, LibMD consistently achieves higher F1 scores than other methods. This is due to the ability of LibMD to leverage code dependency features at multiple scales, including class, function, and basic block levels. By enhancing the coupling between dependencies across these scales, LibMD preserves the usability of its features even when code obfuscation alters or removes portions of the code, demonstrating strong resistance to various obfuscation techniques.

(3) When the obfuscation type is control flow randomization (Ctrl), the precision, recall, and F1 score of LibMD are all higher than those of other methods. The reason is that LibMD can effectively maintain the stability and availability of CFG features before and after obfuscation by integrating contextual instruction semantics during the CFG feature extraction process at the basic block scale.

(4) In cases where code obfuscation results in minor modifications to an app, extracting opcode features for similarity calculation can provide greater accuracy. For example, with Rmv obfuscation type, LibScan shows higher precision on the Zhan dataset compared to LibMD. However, opcode features display lower stability across different app compilation environments, decompilation tools, or varied obfuscation types. For instance, the recall and F1 score of LibScan on the Huang dataset are significantly lower than those of LibMD. By extracting a range of dependency features, including classes, functions, and basic blocks, LibMD enhances detection precision and improves feature stability across diverse obfuscation scenarios, achieving reliable detection performance.

4.5.3. Balancing detection performance and efficiency

To assess the statistical significance of the improvements achieved by LibMD compared to state-of-the-art approaches, we conducted a Wilcoxon Signed-Rank Test on the performance metrics, including Precision, Recall, and F1 scores, across multiple datasets. This non-parametric test is chosen as it is well-suited for comparing paired samples and does not assume normality of the data. The test results indicate that the observed improvements are statistically significant at a 95% confidence level ($p < 0.05$). Specifically, our method consistently outperforms baseline approaches on all metrics across the datasets, confirming the robustness and reliability of LibMD.

Table 7
Ablation experiment results on the dataset without code obfuscation.

Dataset		LibMD-CD	LibMD-LD	LibMD
Wang	P	92.88%	76.21%	**93.23%**
	R	97.82%	98.46%	**99.10%**
	F1	95.29%	85.92%	**96.08%**
Huang	P	73.93%	69.46%	**74.42%**
	R	93.28%	93.07%	**93.71%**
	F1	82.49%	79.55%	**82.96%**
Average	P	83.41%	72.84%	**83.83%**
	R	95.55%	95.77%	**96.41%**
	F1	88.89%	82.73%	**89.52%**

The non-obfuscated part of Huang dataset is used for testing; P: precision; R: recall.

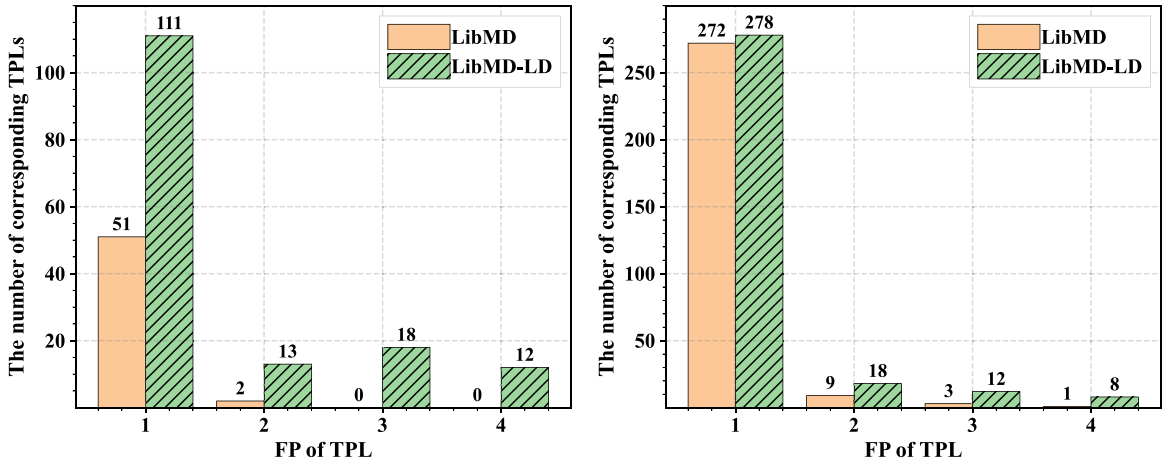
LibMD employs a multi-scale dependency extraction approach combined with a library dependency graph. These techniques enhance detection performance by capturing both fine-grained and large-scale code relationships, which are robust against various obfuscation techniques. However, these methods introduce additional computational steps, such as multi-scale feature extraction and posterior probability computation. To mitigate computational costs, LibMD integrates optimized similarity computations using Bloom filters and the Kuhn-Munkres algorithm. The Bloom filters efficiently encode package- and class-level feature vectors, minimizing memory usage and avoiding exhaustive string matching. Meanwhile, the Kuhn-Munkres algorithm ensures optimal class matching while reducing computational overhead. As a result, LibMD balances detection performance and efficiency, achieving high precision, recall, and F1 scores even on large and complex datasets.

4.6. Ablation study

To evaluate the impact of different modules on the performance of LibMD, ablation studies are conducted on two key components: (1) Code dependency extraction module (CD): In this experiment, only class-scale features are retained during the code dependency feature extraction process, while function- and basic block-scale features are omitted. The detection performance of LibMD is compared with its variant, LibMD-CD, to assess the contribution of the CD module to TPL detection. (2) Version identification module (LD): In this setting, the candidate TPL list is directly generated through the similarity calculation module, bypassing the version identification process. The detection performance of LibMD is compared with its variant, LibMD-LD, to analyze the effect of the LD module on TPL detection. All experiments are conducted using LibMD, LibMD-CD, and LibMD-LD with a similarity threshold of 0.6.

The results of TPL detection using methods LibMD, LibMD-CD, and LibMD-LD on the Wang, Zhan, and Huang datasets are presented in Tables 7, 8. The effectiveness of the version identification module in reducing erroneous versions on the Wang and Huang datasets (for the sections without code obfuscation) is illustrated in Fig. 9. The horizontal axis represents the number of false positive (FP) versions identified within each TPL, while the vertical axis indicates the number of TPLs with version identification errors. For example, in Fig. 9(b), the first bar represents an FP count of $51 \times 1 = 51$. The comprehensive analysis of the experimental results is as follows:

(1) The code dependency extraction module significantly enhances the obfuscation resistance of LibMD and improves detection recall. On the Zhan and Huang datasets, which include obfuscated code, LibMD achieves a 2.35% increase in average recall compared to LibMD-CD. Additionally, LibMD demonstrates notable improvements in recall rates for ALW and ALS obfuscation types, particularly under Fltn, Ctrl, Rmv, PGD, PGR, PGF, DOR, and DOF obfuscation techniques. Notably, the obfuscators ProGuard and DashO perform a greater degree of dead code removal compared to Allatori, highlighting the ability of this module



(a) Wang dataset

(b) Huang dataset (non-code obfuscated portion)

Fig. 9. Reduction in false positives on two datasets.

Table 8
Ablation experiment results on the dataset with code obfuscation.

Dataset	Type		LibMD-CD	LibMD-LD	LibMD
Zhan	Fltn	P	92.06%	77.44%	**96.94%**
		R	95.63%	96.33%	**97.54%**
		F1	93.81%	85.86%	**97.24%**
	Ctrl	P	92.59%	78.85%	**98.16%**
		R	97.74%	97.74%	**98.77%**
		F1	95.10%	87.28%	**98.46%**
	Rmv	P	92.06%	77.87%	**92.24%**
		R	93.02%	93.84%	**94.66%**
		F1	92.54%	85.11%	**93.43%**
Huang	PGD	P	79.05%	75.76%	**79.49%**
		R	47.01%	50.21%	**50.32%**
		F1	58.96%	60.39%	**61.63%**
	PGR	P	65.41%	62.58%	**66.21%**
		R	36.78%	39.77%	**40.94%**
		F1	47.08%	48.63%	**50.60%**
	PGF	P	69.84%	68.73%	**70.63%**
		R	46.06%	48.61%	**49.15%**
		F1	55.51%	56.95%	**57.96%**
	ALW	P	65.95%	64.30%	**67.64%**
		R	80.28%	80.38%	**81.02%**
		F1	72.41%	71.45%	**73.73%**
	ALS	P	49.27%	48.74%	**55.34%**
		R	85.50%	85.82%	**86.57%**
		F1	62.52%	62.17%	**67.52%**
	DOR	P	62.46%	60.29%	**64.03%**
		R	36.46%	39.98%	**40.30%**
		F1	46.04%	48.08%	**49.47%**
	DOF	P	69.33%	68.69%	**69.62%**
		R	46.59%	48.93%	**49.36%**
		F1	55.73%	57.15%	**57.77%**
Average		P	73.80%	68.33%	**76.03%**
		R	66.51%	68.16%	**68.86%**
		F1	67.97%	66.31%	**70.78%**

The obfuscated part of Huang dataset is used for testing; P: precision; R: recall.

to enhance feature coupling. Even when code obfuscation alters or removes portions of the code, LibMD maintains the usability of its features and demonstrates robust resistance to obfuscation.

(2) The version identification module (LD) significantly enhances the version identification capability of LibMD and improves detection accuracy. Compared to LibMD-LD, LibMD achieves a 17.02% and 4.96% increase in detection precision on the Wang and Huang datasets (without code obfuscation), respectively. Fig. 9 illustrates the effectiveness of the LD module in reducing version misidentifications. Without the LD module, the number of false positives (FP) in TPL version identification for these two datasets is 239 (calculated as $111 \times 1 + 13 \times 2 + 18 \times 3 + 12 \times 4$) and 382 (calculated as $278 \times 1 + 18 \times 2 + 12 \times 3 + 8 \times 4$), respectively. With the LD module, FP is reduced to 55 and 303, representing performance improvements of 76.99% and 20.68%, respectively. Furthermore, in apps with code obfuscation, the detection recall rate of LibMD improves by 1.02% and 0.57%. This demonstrates that constructing a library dependency subgraph with correctly identified TPLs as root nodes further enhances the obfuscation resistance of LibMD and reduces the instances of multiple versions being misidentified, particularly those corresponding to horizontal coordinates of 2, 3, and 4.

4.7. Case study of LibMD for TPL detection

To demonstrate the practical utility of LibMD in real-world scenarios, we conduct a case study using Android apps from the AndroZoo [49] repository. Specifically, 10 apps published in 2023 are randomly selected and analyzed using LibMD to detect the presence of TPLs and identify their respective versions. The analysis revealed that 6 out of the 10 apps contained vulnerable TPLs. For these libraries, we provide detailed information on the associated security vulnerabilities, including Common Vulnerabilities and Exposures (CVE) identifiers, severity ratings, as summarized in Table 9.

The detection process involved three critical steps. First, LibMD extract multiscale features from the apps, including class references, function invocations, and basic block control flows, to facilitate accurate TPL detection. Second, library versions are identified using Bayesian probability calculations, which combined metadata dependencies with code similarity measures. Finally, the detected libraries are cross-referenced with publicly available vulnerability databases to identify known security issues and assess their potential impact.

For example, one of the analyzed apps is found to include an outdated version of Library gson (version 2.6.2), which is associated with CVE-2022-25647, a high-severity vulnerability with a CVSS score of 7.5. This vulnerability poses a risk of DoS attacks. Similarly, another app include Library commons-io (version 2.6), linked to CVE-2021-29425, a medium-severity vulnerability with a CVSS score of 5.8, which could lead to “limited” path traversal.

This case study underscores the capability of LibMD not only to detect and identify TPLs with high precision but also to uncover security vulnerabilities in these libraries. These findings highlight the practical applicability of LibMD for real-world software security analysis, particularly in identifying and mitigating risks associated with vulnerable third-party components.

Table 9
Case study of LibMD for TPL detection.

App	TPL	Vulnerabilities	CVSS
1	gson-2.6.2	CVE-2022–25647	High
	retrofit-2.2.0	CVE-2018–1000850	High
		CVE-2018–1000844	Serious
2	gson-2.8.2	CVE-2022–25647	High
	databind-2.11.1	CVE-2022–42004	High
		CVE-2022–42003	High
		CVE-2022–42003	High
		CVE-2020–36518	High
3	gson-2.2.4	CVE-2022–25647	High
	commons-io-2.6	CVE-2021–29425	Medium
4	gson-2.6.2	CVE-2022–25647	High
	retrofit-2.2.0	CVE-2018–1000850	High
		CVE-2018–1000844	Serious
5	fastjson-1.2.11	CVE-2022–25845	Serious
		CVE-2017–18349	Serious
6	commons-io-2.6	CVE-2021–29425	Medium

5. Discussions

The proposed LibMD method functions as an offline analysis framework, ensuring that it operates independently of system-level processes, including process scheduling algorithms. This independence allows for seamless integration into existing workflows without impacting runtime performance or system stability. By focusing on the detection and version identification of Android TPLs across various obfuscation scenarios, LibMD achieves notable improvements in both detection precision and robustness. Nonetheless, certain limitations persist, highlighting opportunities for further research and potential enhancements.

First, LibMD relies heavily on extracting code dependency features such as class references, function invocations, and basic block control flows. This reliance makes it less effective for libraries that primarily consist of resource files or configurations, such as UI libraries. These libraries lack sufficient code dependencies, resulting in challenges in accurate detection. Second, distinguishing between closely related library versions remains difficult due to their high structural and semantic similarity. While posterior probability calculations improve version identification, additional methods to refine semantic or structural analysis could further enhance precision. Third, the computational complexity of processing library dependency graphs and performing similarity calculations may pose scalability challenges in analyzing large-scale apps or highly complex dependency relationships. This could hinder real-time or large-scale deployment scenarios.

The applicability and extensibility of the proposed method are primarily focused on the Android ecosystem due to its reliance on platform-specific characteristics. Specifically, LibMD leverages the Dalvik/ART bytecode format, APK file structure, and common obfuscation techniques that are unique to Android apps. These features allow for the effective extraction of multiscale code dependencies and the construction of library dependency graphs tailored to Android architecture. However, the underlying principles of multiscale feature extraction and dependency graph analysis are not inherently restricted to Android. With appropriate modifications, the method could be adapted to other platforms. For instance, in iOS apps, similar techniques could be employed by analyzing the Mach-O file format and dynamic library dependencies. These extensions would require addressing platform-specific characteristics, such as unique code signing and sandboxing mechanisms of Apple.

In future work, enhancing the applicability and robustness of libMD involves several key directions. Incorporating non-code features such as resource file analysis and metadata patterns could improve detection for a broader range of libraries, while advanced semantic metrics may better distinguish closely related versions. To reduce reliance on POM files, machine learning models trained on API usage patterns, call graphs, and class hierarchies could infer library dependencies even in the absence of explicit metadata. For advanced obfuscation techniques like instruction reordering and code virtualization, integrating dynamic analysis methods, such as runtime behavior monitoring and execution tracing, alongside static analysis, offers a promising hybrid approach. Additionally, optimizing the computational pipeline through parallel processing and memory-efficient algorithms could enhance scalability and reduce overhead for large-scale apps.

One potential application of LibMD is in large-scale app marketplaces, such as the Google Play Store, which host millions of frequently updated apps. In such environments, ensuring the security and compatibility of TPLs is essential. LibMD can be integrated into backend systems to detect outdated or vulnerable libraries across vast datasets, even in the presence of code obfuscation. Its ability to accurately differentiate closely related library versions enables the precise identification of libraries with known vulnerabilities, providing actionable insights for developers and security analysts. Another key application lies in enterprise-level software development and supply chain security. Companies often integrate multiple TPLs into their applications, which introduces risks such as dependency conflicts, security vulnerabilities, and license compliance issues. LibMD can be deployed during the development process to analyze libraries used in projects, ensuring they satisfy security and compatibility requirements. By identifying vulnerable or incompatible library versions, LibMD supports proactive risk management and enhances the reliability of enterprise applications. Furthermore, the modular design of LibMD makes it highly adaptable to specific industrial contexts. For instance, in sectors such as financial technology or healthcare, where compliance and security standards are exceptionally stringent, LibMD can be configured to prioritize the detection of libraries with compliance-critical or security-sensitive characteristics. This flexibility enables LibMD to satisfy the unique requirements of industries with heightened security and regulatory demands.

6. Conclusions

This paper presents a method for detecting TPLs in Android applications using multi-scale dependency relationships. The proposed method, LibMD, extracts code dependencies from class references, function invocations, and basic block control flows. It establishes compact relational representations tailored to various code obfuscation patterns, effectively reducing feature loss caused by obfuscation and enhancing resistance to it. By mining library dependencies from TPL metadata and constructing a library dependency graph based on the similarity between multiple TPLs and the app, LibMD applies Bayes' theorem to compute the likelihood of a TPL being included in the app, enabling precise version identification. Experimental results demonstrate that the code dependency features extracted by LibMD mitigate the impact of complex obfuscation techniques, such as dead code removal and control flow randomization, enabling the detection of a greater number of TPLs across diverse obfuscation scenarios. Furthermore, the combination of similarity-based and library dependency-based version identification accurately predicts the presence of adjacent TPL versions in the app, significantly reducing misidentifications and improving detection performance, as measured by precision, recall, and F1-score.

While LibMD has demonstrated effectiveness in detecting TPLs under various obfuscation techniques, future work can address certain limitations. Incorporating non-code artifacts, such as resource file analysis and metadata patterns, can broaden its applicability to libraries with minimal code dependencies. Optimizing the computational pipeline with parallel processing and memory-efficient algorithms will improve scalability for large-scale applications. Expanding LibMD to other platforms, such as iOS, by supporting Objective-C and Swift code structures and frameworks like CocoaPods, is another promising

direction. Integrating runtime analysis could strengthen its robustness against advanced obfuscation techniques, such as dynamic code generation, while advanced semantic analysis and refined similarity metrics may enhance its ability to differentiate closely related library versions. These advancements aim to transform LibMD into a versatile and robust tool for cross-platform TPL detection in real-world scenarios.

CRediT authorship contribution statement

Zhao Zhang: Writing – review & editing, Writing – original draft, Conceptualization. **Senlin Luo:** Supervision, Funding acquisition, Conceptualization. **Yongxin Lu:** Validation, Software, Methodology. **Limin Pan:** Project administration, Investigation, Data curation.

Declaration of competing interest

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Acknowledgments

This work is supported in part by the 242 National Information Security Projects, PR China (no. 2020A065), the 2020 Information Security Software Project of the Ministry of Industry and Information Technology, PR China (no. CEIEC-2020-ZM02-0134).

Data availability

The authors do not have permission to share data.

References

[1] T. Mahmud, M. Che, G. Yang, Detecting android API compatibility issues with API differences, IEEE Trans. Softw. Eng. 49 (2023) 3857–3871, http://dx.doi.org/10.1109/TSE.2023.3274153.

[2] E. Derr, S. Bugiel, S. Fahl, Y. Acar, M. Backes, Keep me updated: An empirical study of third-party library updatability on android, in: Proc. ACM SIGSAC Conf. Comput. Commun. Secur, 2017, pp. 2187–2200, http://dx.doi.org/10.1145/3133956.3134059.

[3] X. Zhan, T. Liu, L. Fan, L. Li, S. Chen, X. Luo, Y. Liu, Research on third-party libraries in android apps: A taxonomy and systematic literature review, IEEE Trans. Softw. Eng. 48 (2022) 4181–4213, http://dx.doi.org/10.1109/TSE.2021.3114381.

[4] Q. He, B. Li, F. Chen, J. Grundy, X. Xia, Y. Yang, Diversified third-party library prediction for mobile app development, IEEE Trans. Softw. Eng. 48 (2022) 150–165, http://dx.doi.org/10.1109/TSE.2020.2982154.

[5] J. Senanayake, H. Kalutarage, M.O. Al-Kadri, A. Petrovski, L. Piras, Android source code vulnerability detection: A systematic literature review, ACM Comput. Surv. 55 (2023) 1–37, http://dx.doi.org/10.1145/3556974.

[6] R. Bhoraskar, S. Han, J. Jeon, T. Azim, S. Chen, J. Jung, S. Nath, R. Wang, D. Wetherall, Brahmastra: Driving apps to test the security of third-party components, in: 23rd USENIX Sec. Symp., USENIX Security, 2014, pp. 1021–1036, https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/bhoraskar.

[7] J. Wang, et al., Understanding malicious cross-library data harvesting on android, in: 30th USENIX Sec. Symp., USENIX Security, 2021, pp. 4133–4150, https://www.usenix.org/conference/usenixsecurity21/presentation/wang-jice.

[8] S. Chen, Y. Zhang, L. Fan, J. Li, Y. Liu, AUSERA: Automated security vulnerability detection for android apps, in: 37th IEEE/ACM Int. Conf. Autom. Softw. Eng. ASE, 2022, pp. 1–5, http://dx.doi.org/10.1145/3551349.3559524.

[9] X. Zhan, L. Fan, T. Liu, S. Chen, L. Li, H. Wang, Y. Xu, X. Luo, Y. Liu, Automated third-party library detection for android applications: Are we there yet? in: Proc. IEEE/ACM 35th Int. Conf. Automated Softw. Eng., ASE, 2020, pp. 919–930, http://dx.doi.org/10.1145/3324884.3416582.

[10] K. Chen, P. Wang, Y. Lee, X. Wang, N. Zhang, H. Huang, W. Zou, P. Liu, Finding unknown malice in 10 seconds: Mass vetting for new threats at the google-play scale, in: 24th USENIX Sec. Symp., USENIX Security, 2015, pp. 659–674, https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/chen-kai.

[11] Y. Shao, X. Luo, C. Qian, P. Zhu, L. Zhang, Towards a scalable resource-driven approach for detecting repackaged android applications, in: 30th Annu. Comput. Sec. Appl. Conf., ACSAC, 2014, pp. 56–65, http://dx.doi.org/10.1145/2664243.2664275.

[12] L. Yu, X. Luo, C. Qian, S. Wang, Revisiting the description-to-behavior fidelity in android applications, in: 23rd Int. Conf. Softw. Anal. Evolut. Reengineering, SANER, 2016, pp. 415–426, http://dx.doi.org/10.1109/SANER.2016.67.

[13] L. Yu, X. Luo, C. Qian, S. Wang, H.K.N. Leung, Enhancing the description-to-behavior fidelity in android apps with privacy policy, Trans. Softw. Eng. 44 (2018) 834–854, http://dx.doi.org/10.1109/TSE.2017.2730198.

[14] Z. Zhang, W. Diao, C. Hu, S. Guo, C. Zuo, L. Li, An empirical study of potentially malicious third-party libraries in android apps, in: 13th ACM Conf. Sec. Priv. Wirel. Mob. Netw, WiSec, 2020, pp. 144–154, http://dx.doi.org/10.1145/3395351.3399346.

[15] Y. Zhang, J. Wang, H. Huang, Y. Zhang, P. Liu, Understanding and conquering the difficulties in identifying third-party libraries from millions of android apps, IEEE Trans. Big Data 8 (2022) 1511–1523, http://dx.doi.org/10.1109/TBDATA.2021.3093244.

[16] Z. Zhang, H. Ma, D. Wu, D. Gao, X. Yi, Y. Chen, Y. Wu, L. Jiang, MtdScout: Complementing the identification of insecure methods in android apps via source-to-bytecode signature generation and tree-based layered search, in: IEEE 9th Eur. Symp. Sec. Pri, EuroS & P, 2024, pp. 724–740, http://dx.doi.org/10.1109/EuroSP60621.2024.00045.

[17] Y. Wu, C. Sun, D. Zeng, G. Tan, S. Ma, P. Wang, LibScan: Towards more precise third-party library identification for android applications, in: 32nd USENIX Sec. Symp, USENIX Security, 2023, pp. 3385–3402, https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yafei.

[18] Y. Wang, H. Wu, H. Zhang, A. Rountev, ORLIS: obfuscation-resilient library detection for android, in: 5th Int. Conf. Mob. Softw. Eng. Syst, MOBILESoft, 2018, pp. 13–23, http://dx.doi.org/10.1145/3197231.3197248.

[19] J. Xu, Q. Yuan, LibRoad: Rapid, online, and accurate detection of TPLs on android, IEEE Trans. Mob. Comput. 21 (2022) 167–180, http://dx.doi.org/10.1109/TMC.2020.3003336.

[20] X. Zhan, T. Liu, Y. Liu, Y. Liu, L. Li, H. Wang, X. Luo, A systematic assessment on android third-party library detection tools, IEEE Trans. Softw. Eng. 48 (2022) 4249–4273, http://dx.doi.org/10.1109/TSE.2021.3115506.

[21] J. Feichtner, C. Rabensteiner, Obfuscation-resilient code recognition in android apps, in: 14th Int. Conf. Availab. Reliab. Sec., ARES, 2019, pp. 1–10, http://dx.doi.org/10.1145/3339252.3339260.

[22] Y. Zhang, J. Dai, X. Zhang, S. Huang, Z. Yang, M. Yang, H. Chen, Detecting third-party libraries in android applications with high precision and recall, in: 25th Int. Conf. Softw. Anal. Evolut. Reengineering, SANER, 2018, pp. 141–152, http://dx.doi.org/10.1109/SANER.2018.8330204.

[23] B. Bichsel, V. Raychev, P. Tsankov, M. Vechev, Statistical deobfuscation of android applications, in: 2016 ACM SIGSAC Conf. Comput. Commun. Sec., CCS, 2016, pp. 343–355, http://dx.doi.org/10.1145/2976749.2978422.

[24] W. Zhou, Y. Zhou, X. Jiang, P. Ning, Detecting repackaged smartphone applications in third-party android marketplaces, in: Second ACM Conf. Data Appl. Sec. Pri., CODASPY, 2012, pp. 317–326, http://dx.doi.org/10.1145/2133601.2133640.

[25] M. Backes, S. Bugiel, E. Derr, Reliable third-party library detection in android and its security applications, in: 2016 ACM SIGSAC Conf. Comp. Commun. Sec., CCS, 2016, pp. 356–367, http://dx.doi.org/10.1145/2976749.2978333.

[26] J. Zhang, A.R. Beresford, S.A. Kollmann, LibID: reliable identification of obfuscated third-party Android libraries, in: 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal, ISSTA, 2019, pp. 55–65, http://dx.doi.org/10.1145/3293882.3330563.

[27] M. Li, P. Wang, W. Wang, S. Wang, D. Wu, J. Liu, R. Xue, W. Huo, W. Zou, Large-scale third-party library detection in android markets, IEEE Trans. Softw. Eng. 46 (2020) 981–1003, http://dx.doi.org/10.1109/TSE.2018.2872958.

[28] J. Huang, B. Xue, J. Jiang, W. You, B. Liang, J. Wu, Y. Wu, Scalably detecting third-party android libraries with two-stage bloom filtering, IEEE Trans. Softw. Eng. 49 (2023) 2272–2284, http://dx.doi.org/10.1109/TSE.2022.3215628.

[29] C.-H. Liu, Z.-J. Zhang, S.-D. Wang, An android malware detection approach using Bayesian inference, in: IEEE Int. Conf. Comput. Inf. Technol, CIT, 2016, pp. 476–483, http://dx.doi.org/10.1109/CIT.2016.76.

[30] X. Liu, Z. Jin, J. Liu, W. Liu, X. Wang, Q. Liu, ANDetect: A third-party ad network libraries detection framework for android applications, in: 39th Annu. Comp. Sec. Appl. Conf. ACSAC, 2023, pp. 98–112, http://dx.doi.org/10.1145/3627106.3627182.

[31] D. Daniel, d. l. C. Alejandro, G. Alessandra, C. Juan, LibKit: Detecting third-party libraries in iOS apps, in: 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng., ESEC/FSE, 2023, pp. 1407–1418, http://dx.doi.org/10.1145/3611643.361634.

[32] J. Tao, J. Shi, M. Fan, Y. Wang, J. Liu, T. Liu, JSLibD: Reliable and heuristic detection of third-party libraries in miniapps, in: 2023 ACM Workshop Sec. Trustworthy Superapps, SaTS, 2023, pp. 11–16, http://dx.doi.org/10.1145/3605762.3624428.

[33] S. Li, et al., LibAM: An area matching framework for detecting third-party libraries in binaries, ACM Trans. Softw. Eng. Methodol. 33 (2023) 1–35, http://dx.doi.org/10.1145/3625294.

[34] J. Wu, Z. Xu, W. Tang, L. Zhang, Y. Wu, C. Liu, K. Sun, L. Zhao, Y. Liu, OSSFP: Precise and scalable C/C++ third-party library detection using fingerprinting functions, in: 45th Int. Conf. Softw. Eng, ICSE, 2023, pp. 270–282, http://dx.doi.org/10.1109/ICSE48619.2023.00034.

[35] M. Li, W. Wang, P. Wang, S. Wang, D. Wu, J. Liu, R. Xue, W. Huo, LibD: Scalable and precise third-party library detection in android markets, in: 39th Int. Conf. Softw. Eng, ICSE, 2017, pp. 335–346, http://dx.doi.org/10.1109/ICSE.2017.38.

[36] Z. Ma, H. Wang, Y. Guo, X. Chen, LibRadar: fast and accurate detection of third-party libraries in android apps, in: 38th Int. Conf. Softw. Eng, ICSE, 2016, pp. 653–656, http://dx.doi.org/10.1145/2889160.2889178.

[37] C. Soh, H.B. Kuan Tan, Y.L. Arnatovich, A. Narayanan, L. Wang, LibSift: Automated detection of third-party libraries in android applications, in: 23rd Asia-Pacific Softw. Eng. Conf, APSEC, 2016, pp. 41–48, http://dx.doi.org/10.1109/APSEC.2016.017.

[38] H. Wang, Y. Guo, Z. Ma, X. Chen, Wukong: a scalable and accurate two-phase approach to android app clone detection, in: 2015 ACM SIGSOFT Int. Symp. Softw. Test. Anal, ISSTA, 2015, pp. 71–82, http://dx.doi.org/10.1145/2771783.2771795.

[39] B. Liu, B. Liu, H. Jin, R. Govindan, Efficient privilege de-escalation for ad libraries in mobile apps, in: 13th Annu. Int. Conf. Mob. Syst. Appl. Serv, MobiSys, 2015, pp. 89–103, http://dx.doi.org/10.1145/2742647.2742668.

[40] A. Narayanan, L. Chen, C.K. Chan, AdDetect: Automated detection of android ad libraries using semantic analysis, in: 9th Int. Conf. Intell. Sens. Sens. Netw. Inf. Process, 2014, pp. 1–6, http://dx.doi.org/10.1109/ISSNIP.2014.6827639.

[41] L. Glanz, S. Amann, M. Eichberg, M. Reif, B. Hermann, J. Lerch, M. Mezini, CodeMatch: obfuscation won't conceal your repackaged app, in: 11th Jt Meet. Found. Softw. Eng, FSE, 2017, pp. 638–648, http://dx.doi.org/10.1145/3106237.310630.

[42] Z. Tang, M. Xue, G. Meng, C. Ying, Y. Liu, J. He, H. Zhu, Y. Liu, Securing android applications via edge assistant third-party library detection, Comp. Sec. 80 (2019) 257–272, http://dx.doi.org/10.1016/j.cose.2018.07.024.

[43] X. Zhan, L. Fan, S. Chen, F. We, T. Liu, X. Luo, Y. Liu, ATVHunter: Reliable version detection of third-party libraries for vulnerability identification in android applications, in: 43rd Int. Conf. Softw. Eng, ICSE, 2021, pp. 1695–1707, http://dx.doi.org/10.1109/ICSE43902.2021.00150.

[44] B.H. Bloom, Space/time trade-offs in hash coding with allowable errors, Commun. ACM 13 (1970) 422–426, http://dx.doi.org/10.1145/362686.362692.

[45] S. Dahlgaard, M. Knudsen, M. Thorup, Practical hash functions for similarity estimation and dimensionality reduction, in: 31st Int. Conf. Neural Inf. Process. Syst, NIPS, 2017, pp. 6618–6628, https://dl.acm.org/doi/pdf/10.5555/3295222.3295407.

[46] S. Almanee, A. Ünal, M. Payer, J. Garcia, Too quiet in the library: An empirical study of security updates in android apps' native code, in: IEEE/ACM 43rd Int. Conf. Softw. Eng, ICSE, 2021, pp. 1347–1359, http://dx.doi.org/10.1109/ICSE43902.2021.00122.

[47] A. Gupta, S. Namasudra, A novel technique for accelerating live migration in cloud computing, Autom. Softw. Eng. 29 (2022) 34, http://dx.doi.org/10.1007/s10515-022-00332-2.

[48] M. Rahman, A. Murmu, P. Kumar, N.R. Moparthi, S. Namasudra, A novel compression-based 2D-chaotic sine map for enhancing privacy and security of biometric identification systems, J. Inf. Sec. Appl. 80 (2024) 103677, http://dx.doi.org/10.1016/j.jisa.2023.103677.

[49] K. Allix, T.F. Bissyandé, J. Klein, Y.L. Traon, AndroZoo: collecting millions of android apps for the research community, in: 13th Int. Conf. Min. Softw. Repos, MSR., 2016, pp. 468–471, http://dx.doi.org/10.1145/2901739.2903508.